Module 4

The AI-Enhanced Product Lifecycle

Module 4
2-hour session

One small feature across the software lifecycle

Learning outcomes

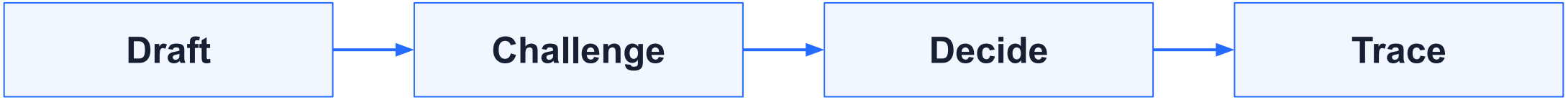
A small change with clear decisions and useful checks

1. Turn an ambiguous request into concrete, testable examples
2. Use AI to draft a diagram and a UI option, then correct them
3. Test one code change against its requirement before fixing it
4. Choose release checks and trace an escaped defect to its cause

The result is a change another person can understand and continue.

Draft, Challenge, Decide, Trace

One requirement, every artifact



Product: agreed examples

Developer: design and change

QA: independent expected results

Operations and support: checks and observed behavior

AI drafts the artifacts. We own whether they agree.

Simulated: the stakeholder request and the support thread. Real: the code, tests and release rehearsal.

An ambiguous feature request

Requirements · live with the assistant

> *“Add deadlines, show overdue work, and perhaps send reminders. Keep it simple and don’t break existing tasks.”*

What does “deadline” mean?

Which day counts as today?

What happens when a date is cleared?

What belongs in the first release?

Look for: one example, one open decision, one deferred item.

Agreed acceptance examples

The contract the class agreed

Rule	Concrete example
Optional date	Old create requests keep working
Omission vs null	A title edit keeps the date. Explicit null clears it.
Strictly before the UTC day	Yesterday is overdue. Today is not.
Completion	Done work never appears in Overdue only.
Persistence and controls	The date survives a GET. Clearing and filtering are visible.

Given a saved date, when I clear it, then a later read has no date.

Where does the change go?

Design · live with the assistant

Ask the assistant for a diagram of how due dates and an Overdue view fit into this application, six boxes at most. Then check it against the real code.

Which components change?

What would be new?

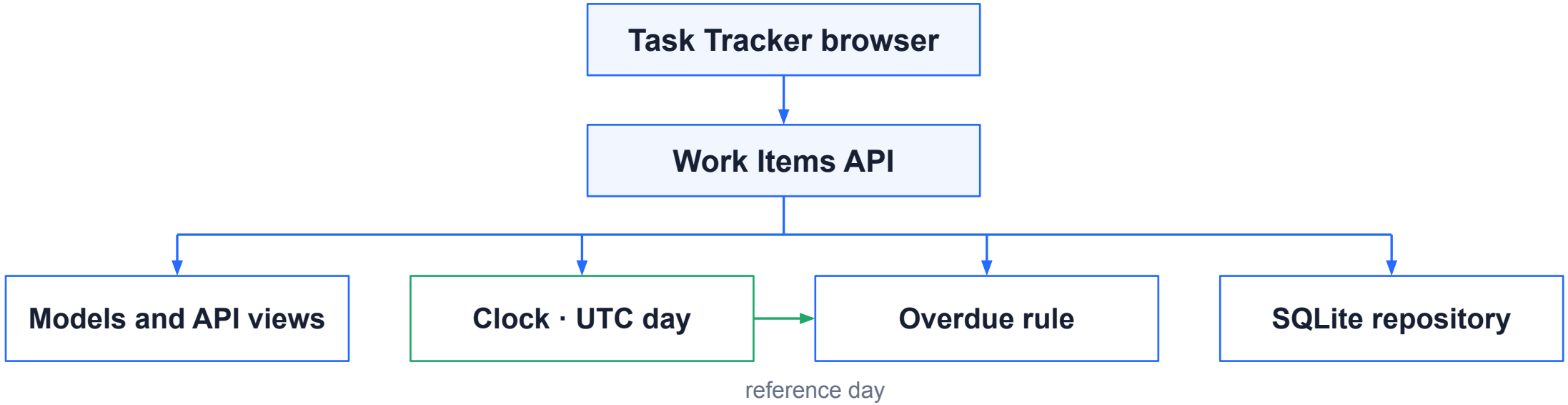
Where does “today” come from?

What does a cleared date pass through?

Look for: one wrong, missing or invented part, and why it matters.

Architecture and the clearing path

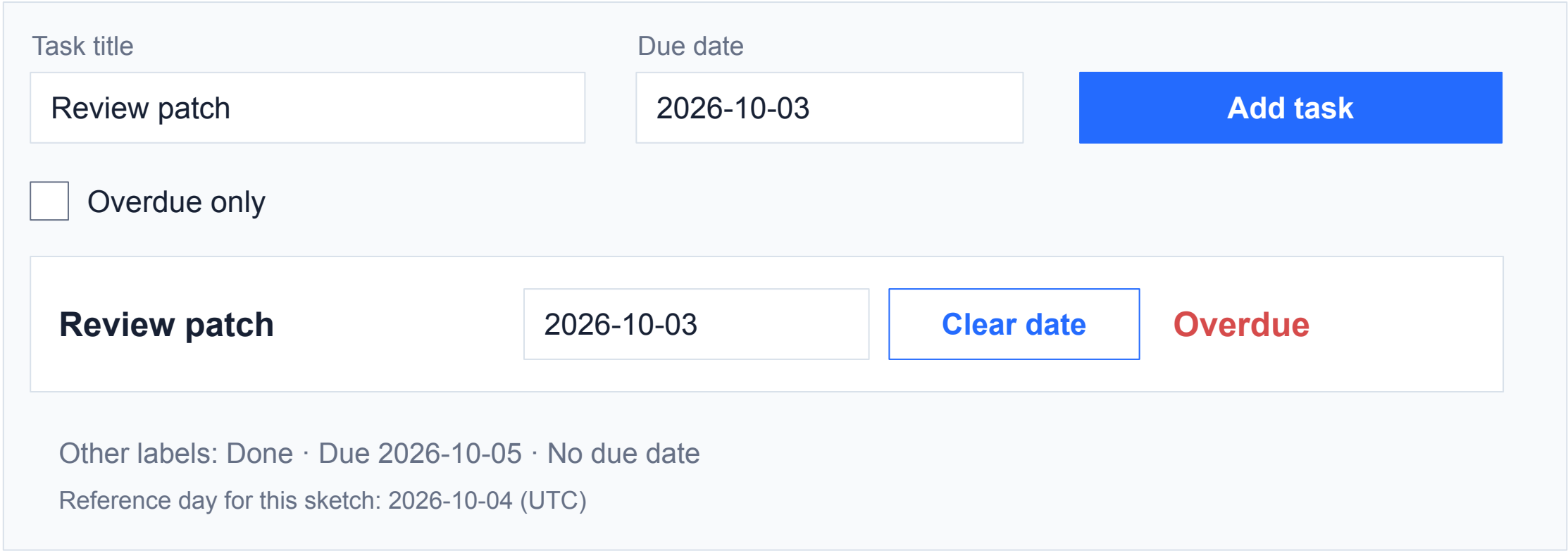
The reference design



Browser sends null. The API keeps the field. The repository saves the cleared date.

A wireframe with real controls

AI drafts two layouts. We check them against the contract.



Check a sketch for a missing control, label or filter before anyone builds it.

Collaborative implementation

The instructor builds the full change. We review the diff together.



Four questions for the diff

Is the migration additive, so old rows survive?	REQ-07
Do the frontend changes stay inside the existing modules?	REQ-08
Is overdue strictly before the reference day, and never for done tasks?	REQ-04 · 05
Does the reference day come from one UTC clock?	REQ-04

Review the diff against the contract, one requirement at a time.

PATCH presence: test first, then fix

Build and test · test-first, live

What the starting code does

```
PATCH /work-items/42  {"due_date": null}
  → 200 OK
GET   /work-items/42
  → "due_date": "2026-10-03"

$ python -m pytest -q
16 passed
```

What we do, in this order

1. Write the clearing test ourselves
2. Run it red for the right reason
3. Ask AI for the smallest fix; read the diff
4. Run the full suite

A supplied null and a missing field carry different instructions.

Useful test cases and independent answers

AI expands the plan. We choose what is worth checking.

Input or action (reference day 2026-10-04)	Expected result	Rule
To do, due yesterday / today	overdue true / false	REQ-04
Complete a past-due task	overdue false	REQ-05
PATCH null, then GET	due_date stays null	REQ-03
Mixed tasks, Overdue only	exactly the intended IDs	REQ-06

A count can be right while the returned task is wrong.

AI and the release pipeline

Today AI writes and reviews the pipeline. Module 6 puts AI inside it.

Now · AI writes and reviews the workflow

```
# AI draft: "test this repo on every push"
jobs:
  tests:
    steps:
      - uses: actions/checkout@v7
      - uses: actions/setup-python@v7
      - run: pip install -r requirements.txt
      - run: python scripts/verify_release.py
```

Module 6 · AI runs inside the pipeline

- Review bots comment on pull requests
- Agents triage failing logs
- Tool-connected agents open fixes

Not in this module: nothing in our workflow calls an AI model.

Review what a workflow runs, not how complete it looks.

Health and business behavior

Release · live with the assistant

Liveness

```
GET /health
  → 200 {"status": "ok"}
```

The process answers.

Business probe · in the release script

```
POST /work-items
  {"status": "done", "due_date": <2 days ago>}
GET /work-items/{id}
  → "overdue": false
GET /work-items?overdue_only=true
  → the probe task is not listed
```

One product rule holds (REQ-05).

Question: name one wrong release this probe would not catch.

A healthy process can still serve the wrong overdue flag.

A noisy support thread

Maintenance · live with the assistant

> *“Overdue labels changed shortly after midnight our time. It happened right after I edited a task.”*

What do the times establish?

Is the saved date wrong?

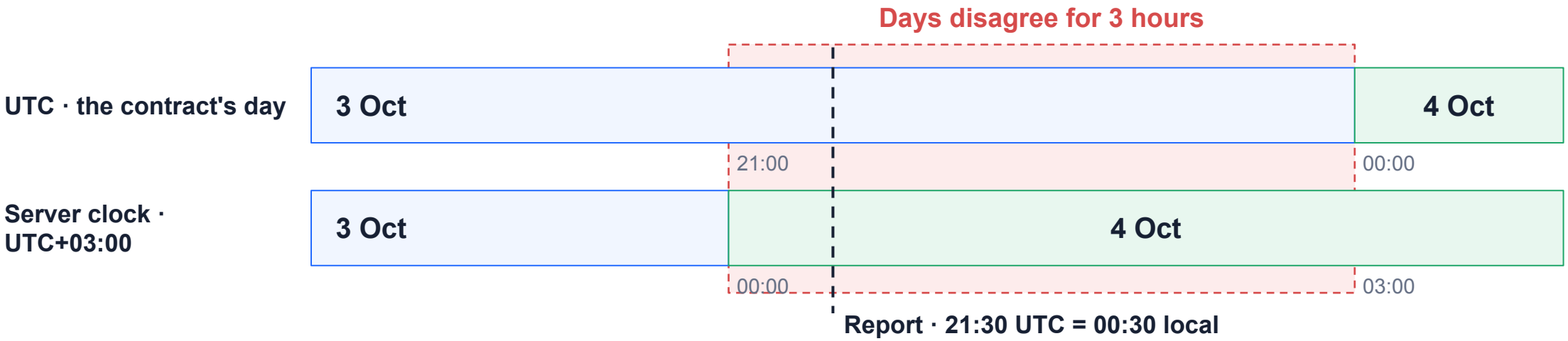
Does a PATCH need to happen?

Which check separates the hypotheses?

Look for: two facts, one hypothesis, one next check.

The clock seam the tests replaced

The same instant falls on two different days



Acceptance fixture

Always returns 4 Oct. Never runs the real clock.

Business probe

A completed task. Passes on any day.

Clock test (next slide)

Replaces only server_now with 00:30 +03:00. Expects 3 Oct.

The release checks passed. The one seam they replaced was never tested.

Close the loop on the escaped defect

Close the loop · live with the assistant

1. Write a test that calls the real get_reference_date() with server_now fixed at 2026-10-04 00:30 +03:00
2. Run it on 0.4.1: red or green, and why?
3. Make the smallest fix in app/main.py; run the full suite
4. Mutation check: change < to <= and record which tests fail

Output

A clock test, the fix, and one mutation result

Review question

Why did 34 passing tests miss this?

Replace only the clock. Never replace the function you are testing.

Fix, handoff and the next decision

The real patch is the input for every message

Verified fix · 0.4.1 → 0.4.2

```
-    return server_now().date()
+    return server_now().astimezone(
+        timezone.utc).date()

+ tests/test_clock_provider.py
  37 passed
```

Handoffs

README: behavior and one example

Internal changelog: cause and new test

User note: the visible change only

Backlog: should “today” follow the team’s timezone?

Maintenance can send a decision back to requirements.

One requirement across the handoffs

A rehearsal for the Module 8 project

REQ-03: Omission keeps the date. Explicit null clears it.

- **Contract:** clearing has a later-read example
- **Design:** the null travels through to storage
- **Code:** PATCH keeps a supplied null
- **Test:** the saved date is gone after GET
- **Release:** README and note use the same example

Product, developer, QA and operations can continue the same change.

Takeaways

The habits that transfer to the next feature

1. AI exposes gaps and drafts useful artifacts quickly
2. Concrete examples make human decisions visible
3. Each check covers a specific part of the change
4. Short handoffs keep the agreed behavior intact

Keep one requirement visible while the work changes hands.

Exit question

5 minutes · one requirement and an escaped mistake · class discussion

Pick one requirement. Where could a mistake be caught, and which checks might still miss it?

1. Name the requirement and a concrete wrong result
2. Point to one check that would catch it
3. Name one check that would miss it, and why

A short, concrete answer is enough.